Journal of Process Control 64 (2018) 49–61

Contents lists available at ScienceDirect

Journal of Process Control

journal homepage: www.elsevier.com/locate/jprocont

Automated feature learning for nonlinear process monitoring – An approach using stacked denoising autoencoder and k-nearest neighbor rule

Zehan Zhang*, Teng Jiang, Shuanghong Li, Yupu Yang

Key Laboratory of Ministry of Education in System Control and Information Processing, Department of Automation, Shanghai Jiao Tong University, Shanghai 200240, China

ARTICLE INFO

Article history:
Received 11 July 2017
Received in revised form 9 February 2018
Accepted 9 February 2018
Available online 20 February 2018

Keywords:
Deep learning
Automated feature learning
Stacked denoising autoencoder
k-Nearest neighbor rule
Nonlinear process monitoring

ABSTRACT

Modern industrial processes have become increasingly complicated, consequently, the nonlinearity of data collected from these systems continues to increase. However, the feature extraction methods of existing process monitoring are not capable of extracting crucial features from these highly nonlinear data, which affects the performance of monitoring. In this paper, a novel nonlinear process monitoring method based on stacked denoising autoencoder (SDAE) and k-nearest neighbor (kNN) rule is proposed. Specifically, stacked denoising autoencoder is utilized to model the nonlinear process data and automatically extract crucial features. The original nonlinear space is then mapped to the feature space and the residual space via SDAE. Two new statistics in the above spaces are constructed by introducing the kNN rule with their corresponding control limits determined by kernel density estimation. Case studies on a nonlinear numerical system and the Tennessee Eastman benchmark process verify the effectiveness of the proposed method.

1. Introduction

Due to the rapid expansion of modern industrial scale, process monitoring has become extremely important in ensuring the regular operation and product quality of industrial systems [1–4]. Large amounts of industrial data have been collected and utilized with distributed control systems (DCSs) widely applied, which leads to widespread use of multivariate statistical process monitoring (MSPM) in the process industries [5–9].

Extracting features and establishing process monitoring model based on extracted features are two major steps of MSPM [10–13]. As to the first step, principal component analysis (PCA) is the most popular feature extraction method [14,15]. PCA, that separates data information into principal component part and residual part, is a linear feature extraction method. The principal component part explains the highest amount of variance of the dataset, while the residual part is less informative. PCA has reached remarkable success in many simple and linear industrial processes. However, the data characteristics of many modern industrial process are actually more complicated, and the relationships among different variables are highly nonlinear. PCA is not powerful enough to extract important features from these data, which results in the poor process monitoring performance [16–18].

In order to address the nonlinear problem of the industrial process and make up for the shortcoming of PCA, several nonlinear methods have been proposed. Kramer [19] proposed a nonlinear PCA method based on the autoassociative neural network. Dong and McAvoy [20] combined the principal curve and the neural network to develop a nonlinear PCA method. However, these neural network based methods did not take into account the effectiveness of the features and are all shallow methods. Another type of the nonlinear methods is the linear approximation approach [21,22]. The idea of this approach is to approximate the nonlinear space by several local linear models. Although the linear approximation approach is easy to implement, the efficiency of nonlinear modeling for this method sometimes can be limited. In recent years, a very popular nonlinear monitoring method based on kernel PCA (KPCA) has been proposed [18,23–25]. KPCA maps the original nonlinear data into a high dimensional linear feature space, and then carries out the linear PCA algorithm in the feature space. The mapping, also called kernel function, is extremely important to the performance of KPCA [26,27]. However, it is difficult to choose a proper kernel function. Up to now, most KPCA methods still use standard kernel functions, such as Gaussian kernel function and polynomial

* Corresponding author.
E-mail address: zehanzhang@126.com (Z. Zhang).

https://doi.org/10.1016/j.jprocont.2018.02.004
0959-1524/

kernel function. In fact, a standard kernel function not always guarantees good results, because it may not be capable of reflecting the real characteristics of the original data [28]. In summary, most of these nonlinear methods are unable to learn useful and crucial features from the nonlinear process data, and thus fail to ensure good monitoring results.

To address the problem mentioned above and achieve automated key features extraction, the present paper proposes a novel nonlinear process monitoring method based on stacked denoising autoencoder (SDAE) [29,30] and k-nearest neighbor (kNN) rule. SDAE, an advanced deep neural network structure, is utilized to model the process data. It is an unsupervised learning method and is often used for feature extraction and tackling nonlinear problems [29,31,32]. More specifically, SDAE applies a local denoising criterion to performing a layer-wise pretraining procedure, stacks consecutive layers together, and obtains the desired deep neural network model by global fine-tuning. Through denoising training and stacked initialization, SDAE is able to obtain a deeper network structure and learn the key features. Benefit from this, SDAE inherits the property of approximating any nonlinear manifold from standard deep neural network [33], which makes it more easier to dig the nonlinear structure and learn important features within process data [34]. Compared to the nonlinear methods mentioned above, the architecture of SDAE is all learned from data, which not only avoids the problem of choosing nonlinear function manually, but also ensures that the learned model truly reflects the data characteristics. Due to the above merits, SDAE has achieved remarkable results in feature extraction and pattern classification [35]. In addition, k-nearest neighbor (kNN) rule is incorporated into SDAE model to perform process monitoring. Specifically, for a new unlabeled sample, the kNN rule first finds the k nearest samples in the training set, and then the category of the new sample is determined by the most frequent category in the k nearest neighboring samples. Here, in detail, SDAE is developed in this study to model the process normal data. The trained SDAE model then converts the original data space to the feature space and the residual space. In order to detect fault, two monitoring charts of $(HD)^2$ and $(RD)^2$ are respectively constructed by utilizing the kNN rule in the feature and residual spaces with their control limits estimated by kernel density estimation (KDE).

The main contribution of this paper can be summarized as follows: (1) An automated key feature learning method is introduced for nonlinear process monitoring. (2) The kNN strategy is combined with the above automated feature learning approach for fault detection. The rest of this paper is arranged as follows. The denoising autoencoder, kNN rule, and KDE are presented in Section 2. Section 3 first introduces the training procedure of SDAE model, which is then followed by the detailed monitoring strategy of the SDAE-kNN method. The method of tuning parameters and validating model is also given. In Section 4, the proposed method is applied to a nonlinear numerical process and the Tennessee Eastman (TE) benchmark process to demonstrate its effectiveness. Finally, Section 5 discusses the conclusions and potential future works.

2. Preliminaries

This section provides an overview of the denoising autoencoder, the kNN rule, and the kernel density estimation for the proposed SDAE-kNN method.

2.1. The denoising autoencoder (DAE)

The motivation of DAE comes from extracting robust and crucial features from partially destroyed inputs [30]. If the input data is corrupted by some random noise, DAE has the ability to undo this corruption and recover the uncorrupted data, which means DAE has learned crucial features. Here, crucial features indicate that DAE is capable of capturing the internal stable structures according to the dependencies and regularities characteristic of the inputs' distribution. The detail of the DAE method is described below.

A corrupted data point is received as input by the DAE and the original (uncorrupted) data point is the output during the DAE training procedure (Fig. 1 depicts the schematic diagram of the procedure). The whole process is as follows [29]:

1. The original input $\mathbf{x}$ is partially corrupted into $\tilde{\mathbf{x}}$ by way of a stochastic mapping $\tilde{\mathbf{x}} \sim q_D(\tilde{\mathbf{x}} \mid \mathbf{x})$.
2. The corrupted input $\tilde{\mathbf{x}}$ is mapped to a hidden representation $\mathbf{h}$ via a deterministic mapping:

$$\mathbf{h} = f(\tilde{\mathbf{x}}) = s(\mathbf{W}\tilde{\mathbf{x}} + \mathbf{b}) \tag{1}$$

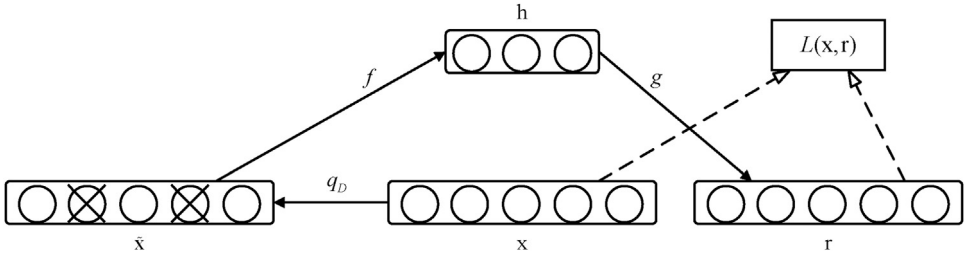

Fig. 1. The computational graph of DAE.

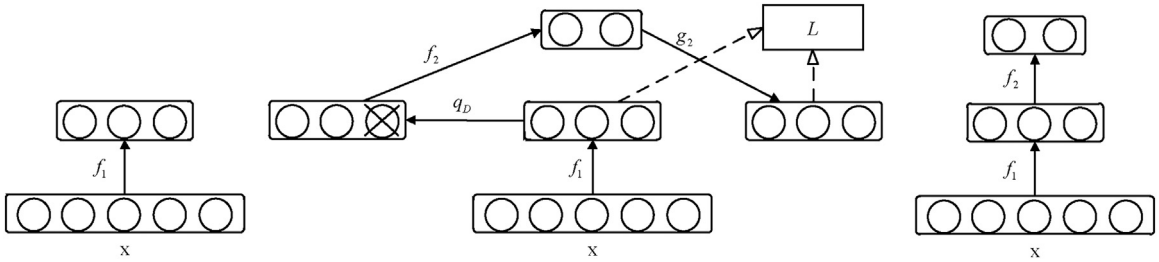

Fig. 2. The training mechanism of SDAE. The left represents half of the first trained DAE. The medium is the training procedure for second DAE. The right is obtained by stacking the encoders of the first and second DAE.

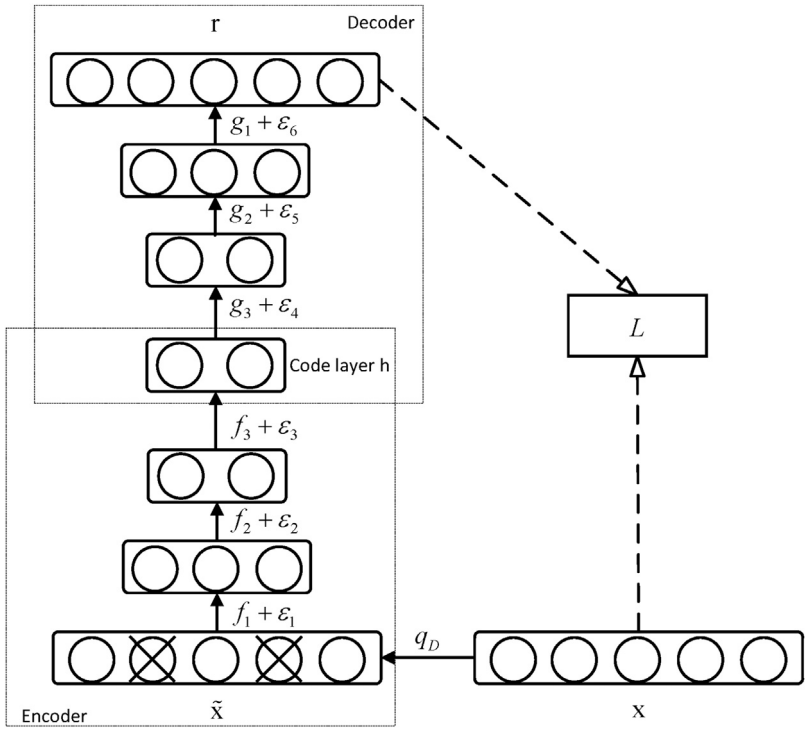

Fig. 3. Fine-tuning of the deep neural network. f_1, f_2, f_3, g_1, g_2 and g_3 come from the stacked pretraining step.

where s is a nonlinear function like the sigmoid function $s(x) = \frac{1}{1+e^{-x}}$, $\mathbf{W}$ is a weight matrix, and $\mathbf{b}$ is the corresponding bias vector.

3. A similar transformation g then maps back the resulting hidden representation $\mathbf{h}$ to a reconstruction denoted by $\mathbf{r}$:

$$\mathbf{r} = g(\mathbf{h}) = s(\mathrm{W}'\mathrm{h} + \mathrm{b}') \tag{2}$$

4. Train the parameters that is $\theta = \{\mathbf{W}, \mathbf{b}, \mathbf{W}', \mathbf{b}'\}$ to minimize the average reconstruction error:

$$\begin{aligned} \theta^* &= \mathrm{argmin}_\theta \frac{1}{n}\sum_{i=1}^{n} L(\mathbf{x}^i, \mathbf{r}^i) \\ &= \mathrm{argmin}_\theta \frac{1}{n}\sum_{i=1}^{n} L(\mathbf{x}^i, g(f(\tilde{\mathbf{x}}^i))) \end{aligned} \tag{3}$$

where n is the number of training samples and L is a loss function that can take many forms according to the distribution assumptions on the input. The conventional one is the squared error $\mathbf{L(x, r)} = \| \mathbf{x} - \mathbf{r} \|^2$. If the interpretation of $\mathbf{x}$ and $\mathbf{r}$ is either bit vector or vector of bit probabilities, the cross-entropy can be an alternative:

$$\mathbf{L}(\mathrm{x}, \mathrm{r}) = -\sum_{k=1}^{d} [\mathbf{x}_k \log \mathbf{r}_k + (1 - \mathbf{x}_k) \log(1 - \mathbf{r}_k)] \tag{4}$$

where d is the dimension of the input $\mathbf{x}$.

Stochastic gradient descent is typically applied to the above optimization, and the required gradients are easily obtained by using the backpropagation algorithm [34].

Note that, the key difference between Denoising Autoencoder and Autoencoder is that $\mathbf{r}$ is now a deterministic mapping of $\tilde{\mathbf{x}}$ rather than $\mathbf{x}$. Each time the stochastic mapping $q_D(\tilde{\mathbf{x}} \mid \mathbf{x})$ is executed, it will generate a different corrupted version $\tilde{\mathbf{x}}$. In this way, DAE no longer learns an identity function but a far more clever mapping that extracts crucial features, unlike the basic autoencoder.

2.2. k Nearest neighbor(kNN) rule

The aim of the kNN rule is classifying a new sample based on similarity with its k nearest neighboring samples in the training set [36]. It is widely utilized in many practical occasions, such as data mining, pattern recognition, and fault detection.

Using kNN rule for fault detection comes from the idea of distance-based anomaly detection [37]: the distance between the fault sample and its k nearest neighbor normal samples is much larger than the distance between the normal sample and its k nearest neighbor normal samples. The process of fault detection using kNN rule is as follows:

1. Finding k nearest neighbors for each normal sample in original normal data space.
2. Calculating the sum of squared distances (D_i^2) for each sample i with its k-nearest neighbors.
3. Determining a threshold (D_α^2) of D^2 for monitoring.
4. Given an unclassified sample $\mathbf{x}$, searching its k nearest neighbors in normal data space with the sum of its squared distances ($D_{\mathbf{x}}^2$) is calculated.
5. Comparing $D_{\mathbf{x}}^2$ to D_α^2: if $D_{\mathbf{x}}^2 \le D_\alpha^2$, $\mathbf{x}$ is normal; otherwise, it is faulty.

2.3. Kernel density estimation (KDE)

KDE, as a non-parametric probability density estimation method [38], was first used by Martin [39] to estimate the control limits of monitoring statistics. Since then, KDE has been widely used in various process monitoring research and applications.

KDE is an effective tool to estimate the probability distribution of observed data. It estimates the probability density function (PDF) by:

$$p(x) = \frac{1}{nh}\sum_{i=1}^{n} K\left(\frac{x - x_i}{h}\right) \tag{5}$$

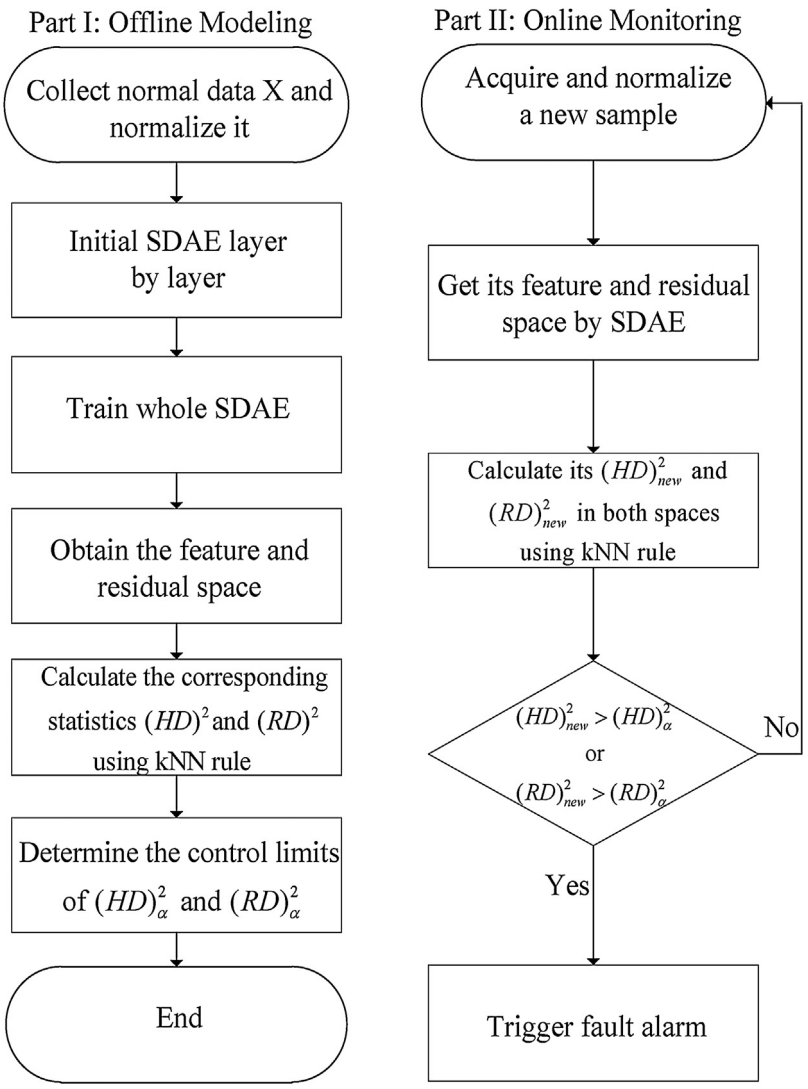

Fig. 4. Flow chart of SDAE-kNN based process monitoring.

where x_i represents the ith sample, n represents the number of observed samples, h represents the bandwidth parameter, and $K(\cdot)$ represents a kernel function satisfying the following condition:

$$\int_{-\infty}^{\infty} K(x)dx = 1 \quad K(x) \geq 0 \tag{6}$$

Among all kinds of kernel density estimation functions, Gaussian kernel function (also called radial basis function) is the most commonly used. Therefore, the Gaussian kernel function is also used in this paper to estimate the probability distribution. In practice, the choice of the bandwidth parameter h has a great influence on the effect of the kernel density estimation. If the selected bandwidth parameter is too small, the estimated PDF will be under-smoothed, however, if the selected bandwidth parameter is too large, the estimated PDF will be over-smoothed. To make a good choice of bandwidth parameters, Mugdadi and Ahmad [40] derived a selection criterion by using the least square cross validation comparison method. The criterion is given by the simple equation below:

$$h = 1.06\sigma n^{-0.2} \tag{7}$$

where σ and n are the standard deviation and the number of sample data, respectively.

After obtaining the PDF of the data by KDE, the cumulative density function (CDF) can be obtained by the following formula:

$$P(x < \theta) = \int_{-\infty}^{\theta} p(x)dx \tag{8}$$

Based on the CDF and the specified confidence level, for example, 99%, the required confidence interval, which is the control limit, can be obtained.

3. Nonlinear process monitoring using stacked denoising autoencoder and kNN rule

This section first introduces the training procedure of SDAE model. The kNN rule is then used in conjunction with the SDAE model to construct two new monitoring statistics for nonlinear process monitoring. The entire monitoring strategy for the proposed method is also given. Finally, some remarks about the model parameters and how to tune them are provided.

3.1. SDAE model development

To automatically find crucial features that truly reflect real data characteristics from complex actual data, establishing SDAE model

Table 1
Detection rates and detection delays (number of samples) of two faults in the nonlinear numerical system.

Fault No.	PCA		KPCA		SDAE-kNN	
	T^2	SPE	T^2	SPE	$(HD)^2$	$(RD)^2$
1	0.000/–	0.408/10	0.027/–	0.982/12	0.986/7	0.998/3
2	0.000/–	0.337/43	0.061/–	0.995/1	1.000/1	1.000/1

in normal data space is a critical step. The development process of SDAE model consists of two parts: stacked pretraining and global fine-tuning.

Stacked pretraining: Given the original data $\mathbf{x}$, DAE, that has only one hidden layer, is first trained to get parameters $[\mathbf{W}_1, \mathbf{W}_1{}', \mathbf{b}_1, \mathbf{b}_1{}']$ and hidden layer output $\mathbf{h}_1 = f_1(\mathbf{x})$. Then $\mathbf{h}_1$ is used as input for the next layer to train the second DAE, which also has only one hidden layer. As before, parameters $[\mathbf{W}_2, \mathbf{W}_2{}', \mathbf{b}_2, \mathbf{b}_2{}']$ and hidden layer output $\mathbf{h}_2$ are obtained. Repeat this process until the code layer is pretrained, and gradually set up a deep neural network. Fig. 2 shows the training process of SDAE. In the deep neural network, this greedy layerwise pretraining process will yield better local minima than randomly initializing the weights.

Global fine-tuning: After pretraining each DAE, a deep neural network is developed by unrolling these DAEs with initialization is performed by using the corresponding pretrained parameters (Fig. 3). The network can be thought of as containing two parts: a deep encoder function and a deep decoder function.

Deep encoder: Get the partially corrupted version $\tilde{\mathbf{x}} \sim q_D(\tilde{\mathbf{x}} \mid \mathbf{x})$. Transform $\tilde{\mathbf{x}}$ into a deep hidden representation $\mathbf{h}$ through a multilayer nonlinear mapping:

$$\mathbf{h} = f_3(f_2(f_1(\tilde{\mathbf{x}}))) = s(\mathbf{W}_3 s(\mathbf{W}_2 s(\mathbf{W}_1 \tilde{\mathbf{x}} + \mathbf{b}_1) + \mathbf{b}_2) + \mathbf{b}_3) \tag{9}$$

Deep decoder: A similar multilayer transformation then maps back the deep hidden representation $\mathbf{h}$ to a reconstruction $\mathbf{r}$:

$$\mathbf{r} = g_3(g_2(g_1(\tilde{\mathbf{x}}))) = s(\mathbf{W}_3{}' s(\mathbf{W}_2{}' s(\mathbf{W}_1{}' \mathbf{h} + \mathbf{b}_1{}') + \mathbf{b}_2{}') + \mathbf{b}_3{}') \tag{10}$$

Finally, the average reconstruction error $\mathbf{L}(\mathbf{x}, \mathbf{r})$ is minimized by using backpropagation algorithm to finetune the whole network. As seen in above training process, all parameters in SDAE model, which are the key factors in the formation of the nonlinear function, are learned from original data. This property ensures that the SDAE model is able to automatically learn the nonlinear relationships among process variables to get the crucial features, which avoids the problem of manually selecting inappropriate nonlinear functions.

Table 3
Process faults in TE process.

Fault No.	Process variable	Type
1	A/C feed ratio, B composition constant (stream 4)	Step
2	B composition, A/C ratio constant (stream 4)	Step
3	D feed temperature (stream 2)	Step
4	Reactor cooling water inlet temperature	Step
5	Condenser cooling water inlet temperature	Step
6	A feed loss (stream 1)	Step
7	C header pressure loss-reduced availability (stream 4)	Step
8	A, B, C feed composition (stream 4)	Random variation
9	D feed temperature (stream 2)	Random variation
10	C feed temperature (stream 4)	Random variation
11	Reactor cooling water inlet temperature	Random variation
12	Condenser cooling water inlet temperature	Random variation
13	Reaction kinetics	Slow drift
14	Reactor cooling water valve	Sticking
15	Condenser cooling water valve	Sticking
16	Unknown	Unknown
17	Unknown	Unknown
18	Unknown	Unknown
19	Unknown	Unknown
20	Unknown	Unknown
21	The valve for stream 4 was fixed at the steady state position	Step constant position

3.2. Incorporating the kNN rule into SDAE for fault detection

After developing the SDAE model, a feature space and a residual space are respectively got via deep encoder and decoder functions. To detect fault, the kNN rule is applied to construct the monitoring statistics in the above spaces, which consists of the following strategies:

1. For each data $\mathbf{x}$ in normal data space $\mathbf{X}$, the feature space $h(\mathbf{X})$ and the residual space $(\mathbf{X} - \mathbf{R}_\mathbf{X})$ are obtained by trained SDAE model, where $\mathbf{R}_\mathbf{X}$ represents the reconstruction space obtained by Eq. (10).
2. Finding k nearest neighbors for $\mathbf{x}$ in both spaces, respectively.
3. Calculating the kNN squared distance of $\mathbf{x}$ in both spaces, respectively:

Table 2
Monitoring variables in TE process.

No.	Measurements	No.	Measurements
1	A feed	18	Stripper temperature
2	D feed	19	Stripper steam flow
3	E feed	20	Compressor work
4	Total feed	21	Reactor cooling water outlet temperature
5	Recycle flow	22	Separator cooling water outlet temperature
6	Reactor feed rate	23	D feed flow valve
7	Reactor pressure	24	E feed flow valve
8	Reactor level	25	A feed flow valve
9	Reactor temperature	26	Total feed flow valve
10	Purge rate	27	Compressor recycle valve
11	Product separator temperature	28	Purge valve
12	Product separator level	29	Separator pot liquid flow valve
13	Product separator pressure	30	Stripper liquid product flow valve
14	Product separator underflow	31	Stripper steam valve
15	Stripper level	32	Reactor cooling water flow
16	Stripper pressure	33	Condenser cooling water flow
17	Stripper underflow		

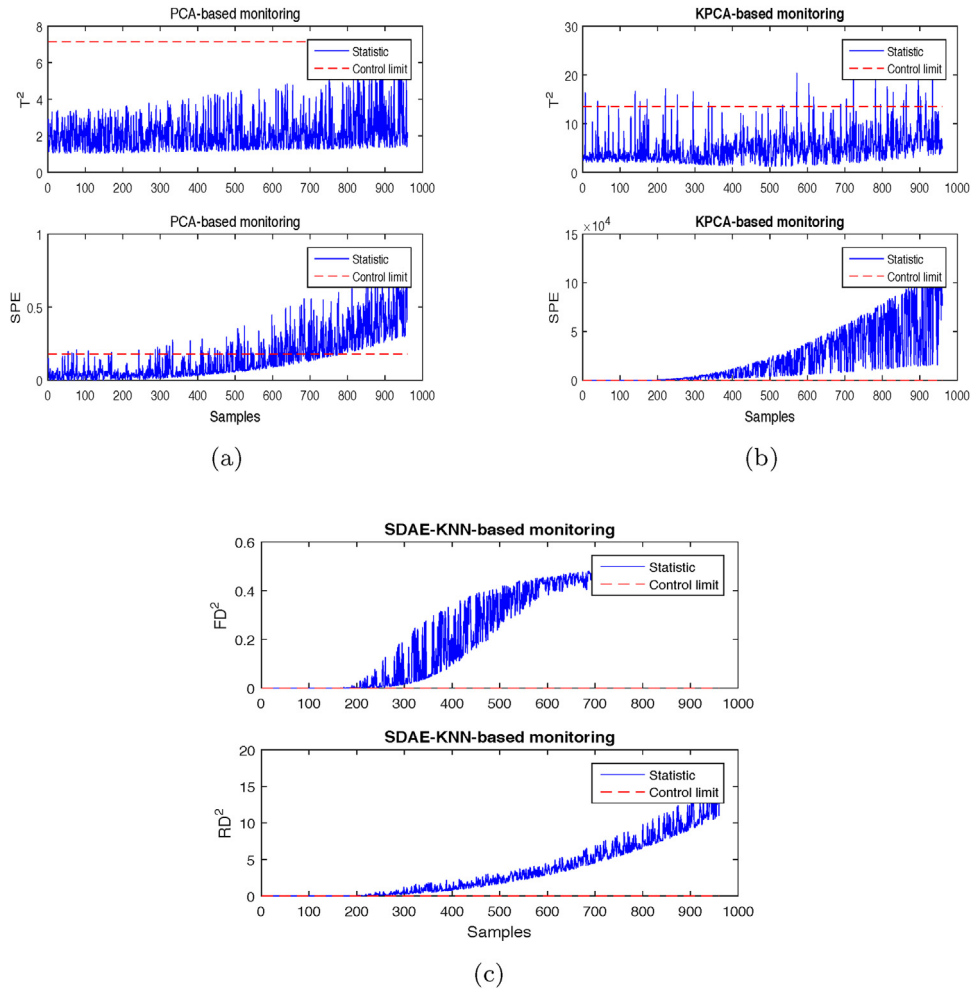

Fig. 5. Monitoring results of fault 1 of the numerical system: (a) PCA, (b) KPCA and (c) SDAE-kNN.

In feature space:

$$(HD)_i^2 = \sum_{j=1}^{k} (hd)_{ij}^2 \tag{11}$$

where $(hd)_{ij}^2$ represents the squared Euclidean distance between sample i and its jth nearest neighbor in feature space.

In residual space:

$$(RD)_i^2 = \sum_{j=1}^{k} (rd)_{ij}^2 \tag{12}$$

where $(rd)_{ij}^2$ represents the squared Euclidean distance between sample i and its jth nearest neighbor in residual space.

4. Determining the thresholds $(HD)_\alpha^2$ and $(RD)_\alpha^2$ by KDE.

When a new sample $\mathbf{x}_{new}$ arrives, $\mathbf{x}_{new}$ is mapped into a feature vector $\mathbf{h}(\mathbf{x}_{new})$ and residual vector $(\mathbf{x}_{new} - \mathbf{r}_{\mathbf{x}_{new}})$ according to the above SDAE model. $(HD)_{\mathbf{x}_{new}}^2$ and $(RD)_{\mathbf{x}_{new}}^2$ can be calculated by using $\mathbf{h}_{\mathbf{x}_{new}}$ and $(\mathbf{x}_{new} - \mathbf{r}_{\mathbf{x}_{new}})$ in the same way as Eqs. (11) and (12). If $(HD)_{\mathbf{x}_{new}}^2 \le (HD)_\alpha^2$ and $(HD)_{\mathbf{x}_{new}}^2 \le (HD)_\alpha^2$, $\mathbf{x}_{new}$ is normal, otherwise, it is faulty.

3.3. Outline of monitoring strategy for the proposed method

Fig. 4 illustrates the complete process monitoring scheme based on SDAE-kNN. The detailed steps are presented as follows.

(1) Offline modeling

Table 4
Parameters for DAEs and SDAE.

Algorithm	Learning rate	Number of epochs	Batch size	Destruction
DAEs	1	600	50	0.001
SDAE	0.1	1000	50	0.1

Step. 1 Get the sample set $\mathbf{X}$ under normal operating conditions and normalize it.

Step. 2 Pretrain the SDAE model by training each DAE and acquire the corresponding initialization weights and bias.

Step. 3 Stack the DAEs obtained in Step 2 to unroll the whole architecture for SDAE.

Step. 4 Finetune the whole SDAE model by minimizing the average reconstruction error.

Step. 5 Obtain the feature and residual spaces using the trained SDAE model.

Step. 6 Construct the monitoring statistics $(HD)^2$ and $(RD)^2$ for $\mathbf{X}$ by incorporating the kNN rule into above spaces, respectively.

Step. 7 Estimate the control limits of $(HD)^2$ and $(RD)^2$ by KDE.

(2) Online monitoring

Step. 1 A new observation $\mathbf{x}_{new}$ is obtained and normalized.

Step. 2 Map $\mathbf{x}_{new}$ into feature space $\mathbf{h}_{\mathbf{x}_{new}}$ and residual space $(\mathbf{x}_{new} - \mathbf{r}_{\mathbf{x}_{new}})$ via the trained SDAE model.

Step. 3 Calculate the values of $(HD)_{\mathbf{x}_{new}}^2$ and $(RD)_{\mathbf{x}_{new}}^2$ in the above two spaces using the kNN rule.

Step. 4 Monitor if $(HD)_{\mathbf{x}_{new}}^2$ or $(RD)_{\mathbf{x}_{new}}^2$ exceeds its corresponding control limit.

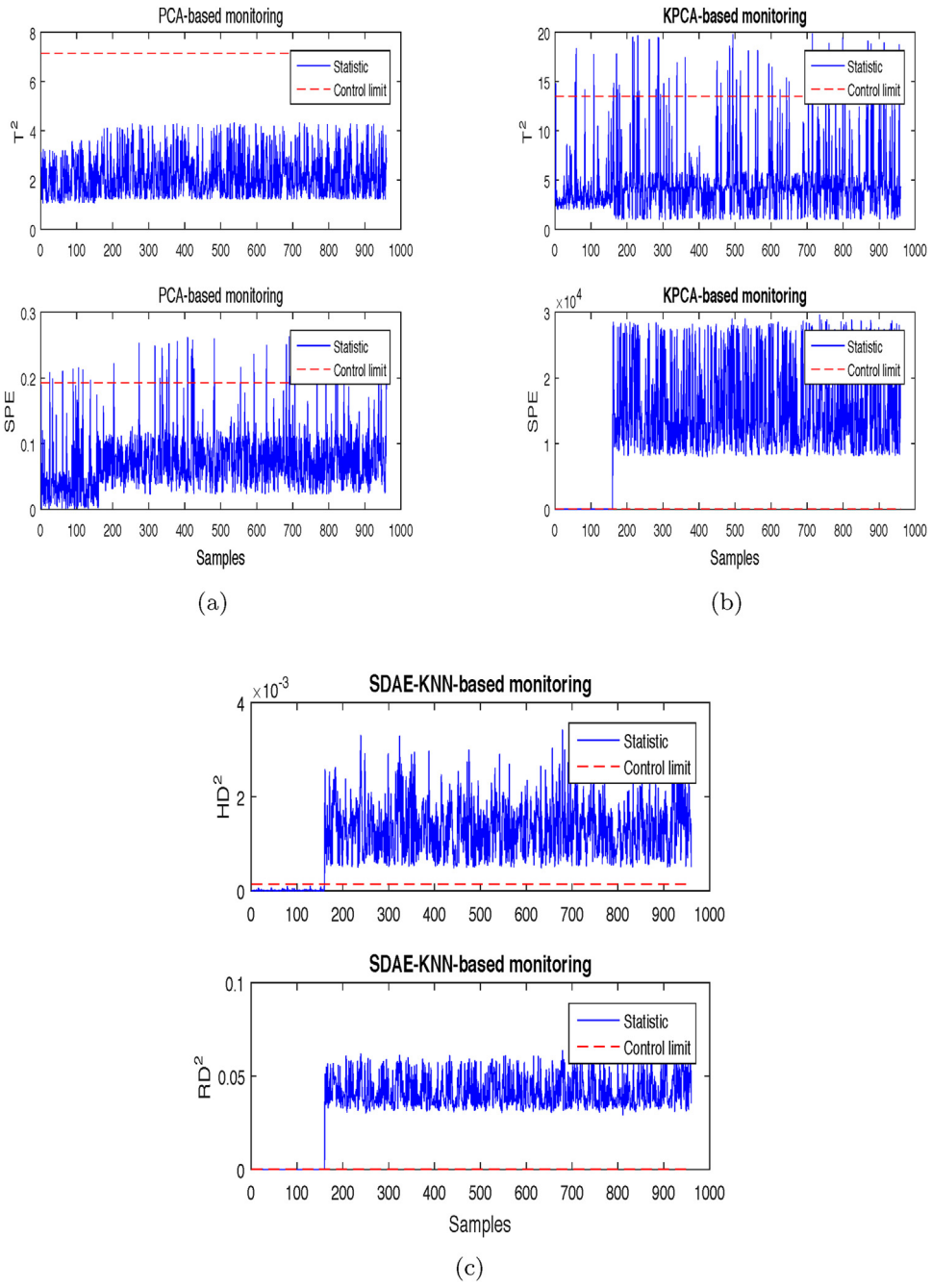

Fig. 6. Monitoring results of fault 2 of the numerical system: (a) PCA, (b) KPCA and (c) SDAE-kNN.

3.4. Some remarks

It is a time-consuming, tedious and iterative process to tune the parameters in SDAE. The major parameters in SDAE include the following:

(1) Number of nodes in any hidden layer.
(2) Number of hidden layers applicable for SDAE.
(3) The masking noise (fraction of the input to be corrupted) in any denoising autoencoder.
(4) Batch size, learning rate and momentum parameter for stochastic gradient descent.
(5) Maximum iterations (number of epochs) to be used for the training.

In order to determine the above parameters, the following criteria are used:

(1) Reconstruction error rate (RER): $\frac{\frac{1}{n}\sum_{i=1}^{n}\|\mathbf{x}^i-\mathbf{r}^i\|^2}{\sum_{i=1}^{n}\|\mathbf{x}^i\|^2}$, which is used to measure how close the reconstruction $\mathbf{r}$ is to the original input $\mathbf{x}$. The smaller the RER, the closer $\mathbf{r}$ is to $\mathbf{x}$.
(2) Fault detection rate (FDR): the proportion of the number of fault samples detected by the algorithm to the actual number of fault samples.

In the actual tuning process, first, we use $RER \leq 0.01$ to roughly determine the number of SDAE layers, the number of nodes per layer, and the masking noise. Then determine the batch size, learning rate, momentum factor, and the maximum iterations according to the FDR in validation data. At the same time, finetune the param-

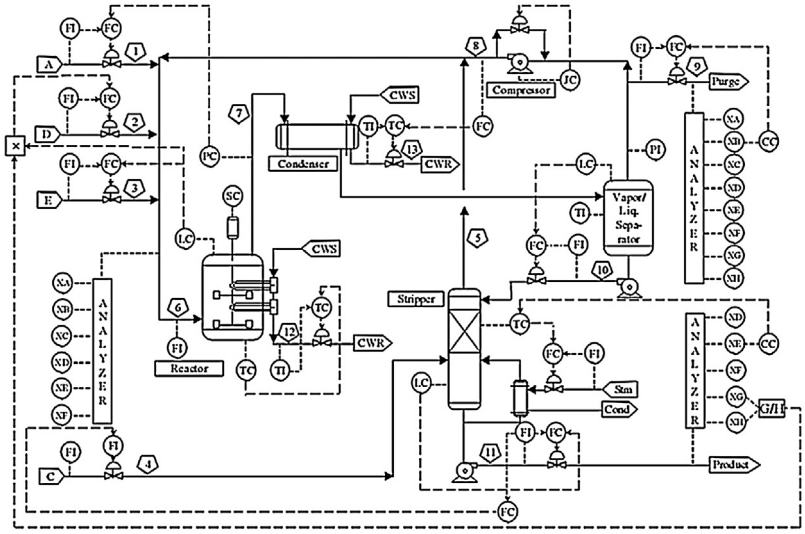

Fig. 7. Control structure of the TE benchmark process [43].

Table 5
Detection rates for all faults in TE process.

Fault No.	PCA		KPCA		SDAE-kNN	
	T^2	SPE	T^2	SPE	$(HD)^2$	$(RD)^2$
1	0.991	1.000	0.997	0.000	0.996	1.000
2	0.960	0.987	0.985	0.003	0.981	0.994
3	0.025	0.041	0.100	0.012	0.053	0.349
4	0.038	0.993	1.000	0.001	0.996	1.000
5	0.217	0.281	0.331	0.035	0.277	0.569
6	0.992	1.000	1.000	0.000	0.996	1.000
7	0.398	1.000	1.000	0.023	1.000	1.000
8	0.933	0.975	0.993	0.037	0.976	0.996
9	0.010	0.043	0.088	0.021	0.062	0.330
10	0.376	0.433	0.522	0.210	0.451	0.873
11	0.203	0.721	0.747	0.026	0.743	0.903
12	0.935	0.977	0.996	0.105	0.986	0.997
13	0.938	0.951	0.952	0.031	0.952	0.968
14	0.825	1.000	1.000	0.000	1.000	1.000
15	0.041	0.051	0.161	0.043	0.065	0.371
16	0.226	0.403	0.376	0.157	0.282	0.901
17	0.737	0.925	0.953	0.007	0.877	0.975
18	0.875	0.902	0.901	0.000	0.901	0.925
19	0.002	0.267	0.081	0.023	0.087	0.761
20	0.250	0.542	0.618	0.066	0.537	0.843
21	0.325	0.467	0.443	0.006	0.510	0.597

Table 6
Detection delays (number of samples) for all faults in TE process.

Fault No.	PCA		KPCA		SDAE-kNN	
	T^2	SPE	T^2	SPE	$(HD)^2$	$(RD)^2$
1	8	1	3	–	4	1
2	26	5	12	17	16	5
3	80	45	10	15	15	4
4	1	1	1	316	1	1
5	16	1	1	57	2	1
6	7	1	1	–	3	1
7	1	1	1	268	1	1
8	27	16	1	229	15	1
9	1	3	1	6	1	1
10	19	6	15	38	6	8
11	7	6	6	181	6	1
12	8	3	3	22	3	3
13	50	39	38	149	38	14
14	2	1	1	–	1	1
15	577	57	92	261	93	2
16	32	16	32	31	2	1
17	29	2	22	61	2	1
18	99	15	61	–	25	4
19	208	11	2	6	42	1
20	79	68	79	79	82	5
21	257	41	251	257	246	12

eters determined in the first step. Repeat the above steps until the optimal parameters are found. The parameters we use will be given later in the experiment.

4. Case studies

This section presents the simulation results in a nonlinear numerical system and the TE benchmark process to demonstrate the effectiveness of the SDAE-kNN method.

4.1. Numerical system

This simple nonlinear system contains five variables, which was suggested by Ge and Song [21]:

$$\begin{aligned} x_1 &= z + e_1, \\ x_2 &= z^2 - 3z + e_2, \\ x_3 &= -z^3 + 3z^2 + e_3, \\ x_4 &= z^4 - 4z^3 + 2z + e_4, \\ x_5 &= -2z^5 + 6z^4 - 3z^3 + z + e_5, \end{aligned} \tag{13}$$

where $z \in [0.01, 2]$, and e_1, e_2, e_3, e_4, e_5 are independent noise of $N(0, 0.01)$. A normal data set with 500 samples is generated by the above formula. In order to test the performance of the SDAE-kNN method, two fault data sets both containing 960 samples are generated:

Fault 1: a ramp change of x_5 by $0.05(k - 160)$ is added to each sample from sample 161 to 960, where k is the sample number;

Fault 2: a step change of x_3 by 1 is introduced from sample 161.

The PCA, KPCA and proposed SDAE-kNN methods are applied to this numerical system. The kernel parameter of KPCA is set to $5m$, where m is the input dimension. The confidence levels of 3 methods are all set as 0.99. Table 1 shows the monitoring results (detection rates/detection delays) of both faults.

The first fault case is a ramp change. Therefore, it is critical to reflect the trend of the fault and detect the fault in time. Fig. 5 shows the monitoring results of the PCA, KPCA and SDAE-kNN methods on fault 1. The conventional PCA method cannot effectively monitor this abnormality, since most of the two monitoring statistics are below the corresponding control limits. Although the fault detection rate of KPCA SPE statistic is 98.25%, the T^2 statistic of KPCA only has 2.75% fault detection rate and the ramp trend of the fault

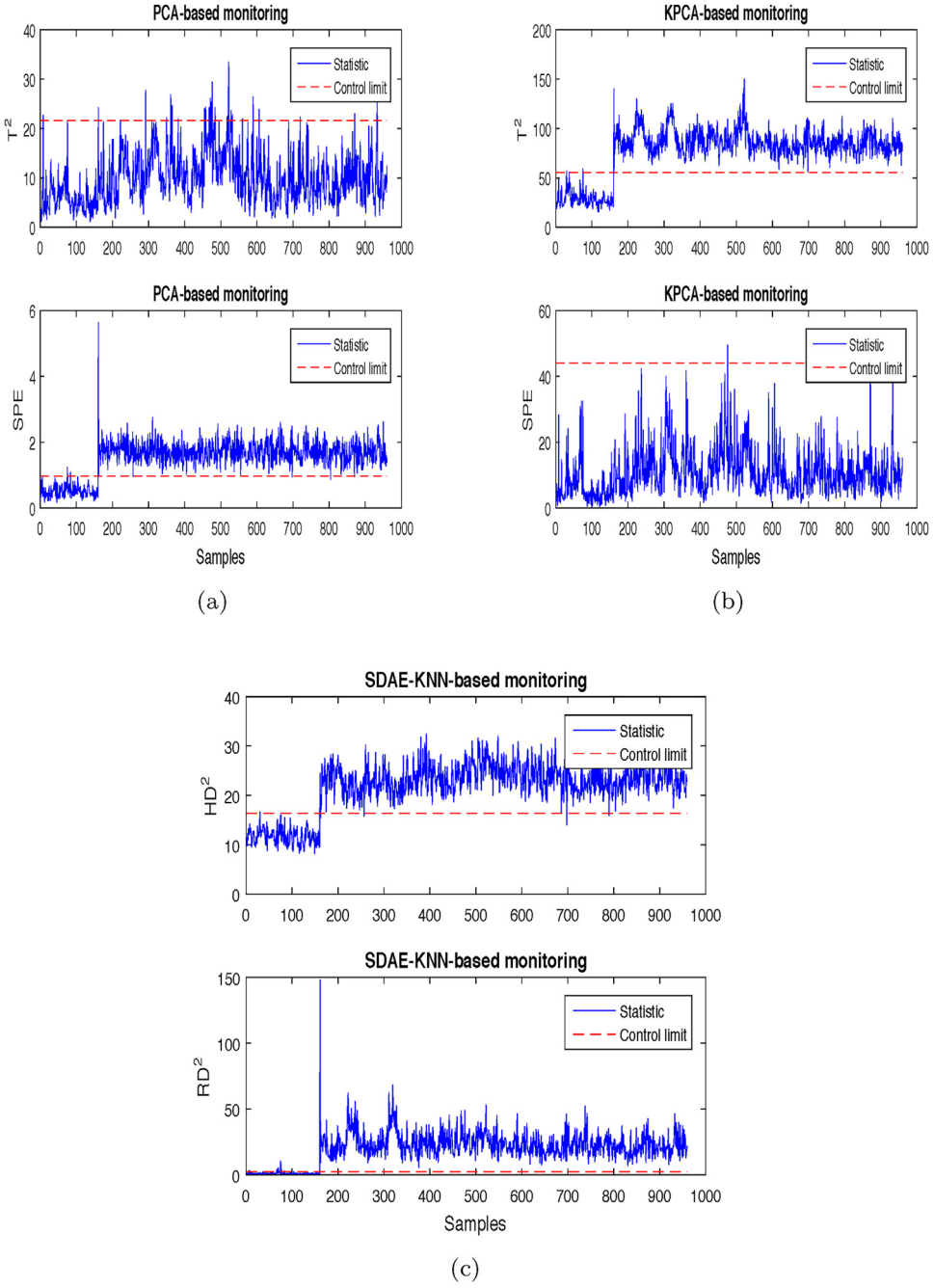

Fig. 8. Monitoring results of fault 4 of the TE process: (a) PCA, (b) KPCA and (c) SDAE-kNN.

is also not detected. The proposed SDAE-kNN method gives the most efficient detection results with the highest detection rates of 98.625% and 99.875%, and the trend of the ramp is also successfully reflected. Besides, for the sensitivity of the fault detection, the two monitoring statistics of the SDAE-kNN method start detecting the fault only at approximately 7th fault sample and 3rd fault sample. Similarly, the monitoring results of fault 2 are illustrated in Fig. 6. The detection rates for the two monitoring statistics of the SDAE-kNN method both reach 100%, outperforming the PCA and KPCA methods. The SDAE-kNN method has capable of finding complex nonlinear relationships and extracting crucial features from the complicated process, which shows the effectiveness in the nonlinear fault detection.

4.2. TE benchmark process

The Tennessee Eastman benchmark process was first developed by Downs and Vogel [41], which was structured by Lyman and Georgakis [42]. The control structure of the TE process is shown in Fig. 7 [43], which consists of five main operating units: a reactor, a condenser, a separator, a compressor, and a stripper. There are 22 continuous measurement variables, 19 composition measurements variables, and 12 manipulated variables. In this study, we select 11 manipulated and all 22 continuous variables for the process monitoring. Table 2 lists the details of above variables. Moreover, Table 3 lists 21 simulated faults, where unknown means that the type and cause of the fault are unknown, unlike the other 16 known faults that are aware of the causes and types. Both normal and fault data

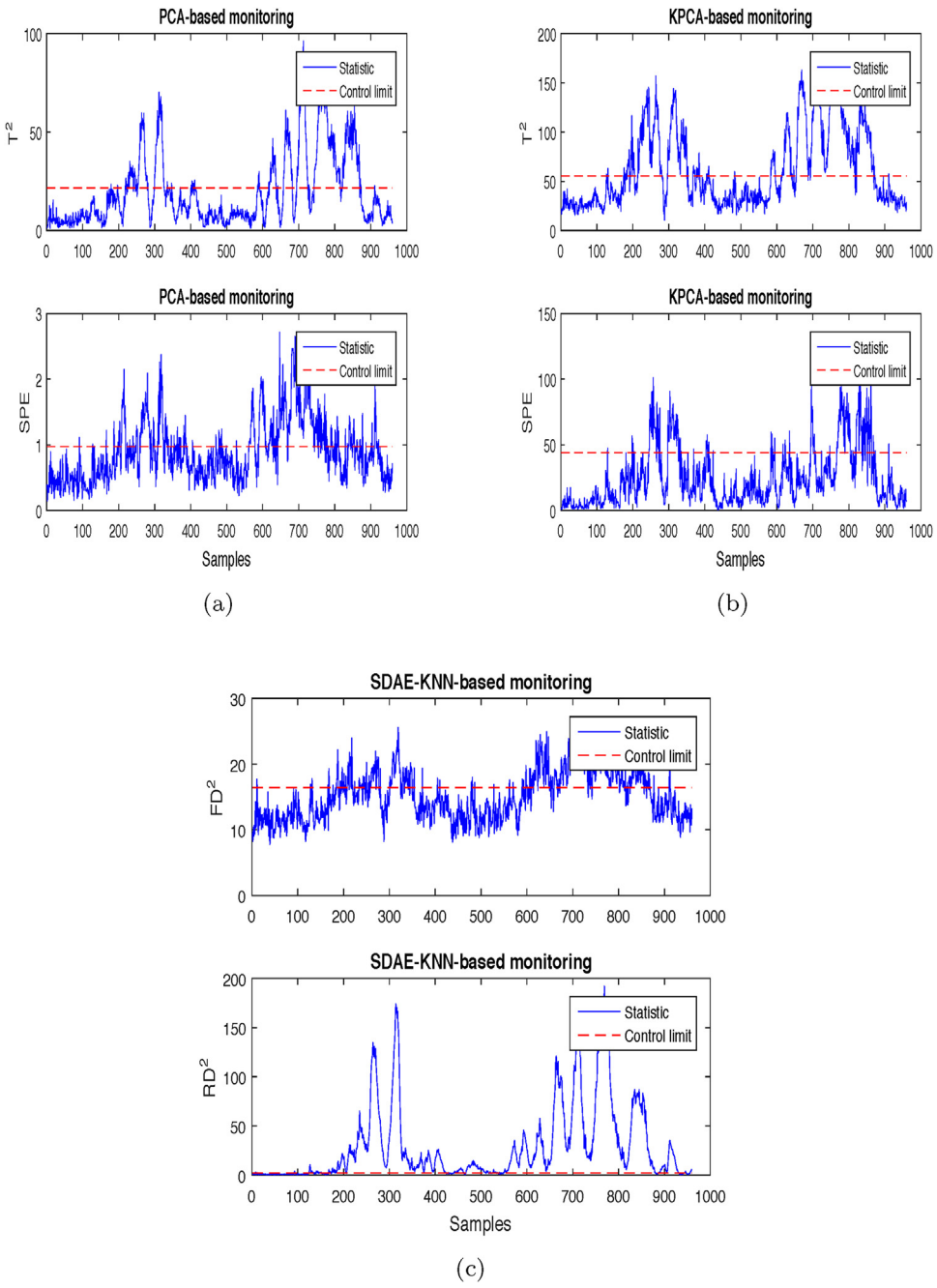

Fig. 9. Monitoring results of fault 10 of the TE process: (a) PCA, (b) KPCA and (c) SDAE-kNN.

sets are generated by the TE process [44]. Each fault data set has 960 samples with a fault introduced from sample 161. The PCA, KPCA and proposed SDAE-kNN methods are developed for comparison. The kernel parameter of KPCA is set to $5m$ and the confidence levels in 3 algorithms are all set as 0.99.

First, the SDAE-kNN model is developed in normal data set followed by obtaining the feature and residual spaces. Meanwhile, the corresponding monitoring statistics for above two spaces are constructed. Please Refer to Section 3.4 for the procedure of tuning parameters. A binary random masking noise is considered as destruction, where a fraction of the input components (randomly chosen) have their value set to zero. The overall network structure of SDAE is [33, 200, 100, 50, 30, 50, 100, 200, 33]. Therefore, the entire SDAE is a stack of four DAEs and the structures of the four DAEs are [33, 200, 33], [200, 100, 200], [100, 50, 100], and [50, 30, 50], which are denoted as DAE_1, DAE_2, DAE_3, and DAE_4. We take the middle layer of SDAE, which has 30 nodes, as the feature layer. Table 4 lists the parameters used in four DAEs and SDAE. The RERs for SDAE model in training and testing data are 0.42% and 0.96%, respectively. Moreover, the control limits of HD^2 and RD^2 estimated by KDE are 16.392 and 2.346.

Table 5 shows the detection rates of the PCA, KPCA, and SDAE-kNN methods. For large magnitude faults 1, 2, 6, 7, 8, 12, 13, 14, and 18, the performance of the SDAE-kNN method is as good as the other methods. All methods cannot adequately detect faults 3, 5, 9, 15 and 21, since the detection rates of them are low. The SDAE-kNN method is seen to be more efficient than the other methods in handling faults 4, 10, 11, 16, 17 and 20. The SDAE-kNN method has higher detection rates on these faults. Table 6 gives the detection delays of three methods. The proposed SDAE-kNN method detects

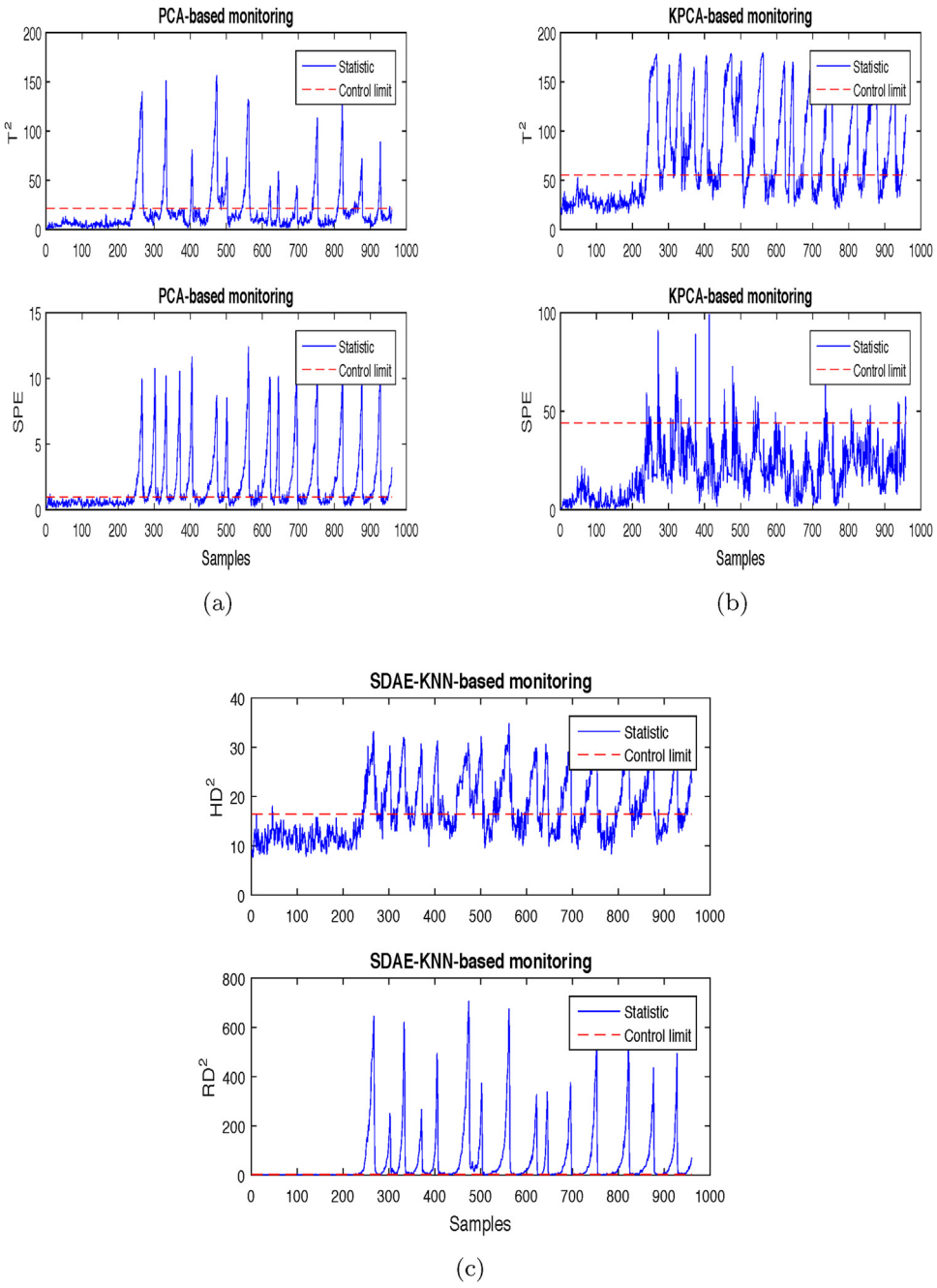

Fig. 10. Monitoring results of fault 20 of the TE process: (a) PCA, (b) KPCA and (c) SDAE-kNN.

faults 2, 10, 11, 13, 16, 17, 18, 20, and 21 more quickly than the other methods. Furthermore, three faults are chosen to illustrate the effectiveness of the SDAE-kNN method in detail.

Fault 4 relates to a step change in the temperature of the reactor inlet cooling water. Compared with the normal state, the change of the variance and mean in each variable are less than 2% [1], which makes the detection of fault 4 more challenging. Comparing the monitoring results shown in Fig. 8, the SDAE-kNN method has the best monitoring results. The PCA and KPCA methods both only have one monitoring statistic to detect the fault. However, the two monitoring statistics of SDAE-kNN both have successfully detected the fault, since almost all values of the two statistics are above the control limits.

Fault 10 is caused by a random temperature change of stream 4. This fault will result in a random variation in process variables, and their variances will become larger. Fig. 9 shows the monitoring results of three methods. As can be seen from the figure, all monitoring statistcs of the PCA and KPCA methods cannot adequately detect the fault, since many statistic values stay below the corresponding control limits. In contrast, the RD^2 statistc of the SDAE-kNN method detects the vast majority of fault samples, which gives the best monitoring result.

Fault 20 is an unknown type fault. Fig. 10 gives the monitoring results. From Fig. 10, it is clear that the SDAE-kNN method has a significant improvement in monitoring performance compared to the other methods. The RD^2 statistc of the SDAE-kNN method has the highest fault detection rate and the lowest detection delay.

Finally, the time required to compute PCA, KPCA, and SDAE-kNN is compared and the detail training time for any step in SDAE is also given. For convenience, the time for developing the model in train-

Table 7
The time required to compute PCA, KPCA, and SDAE-kNN (s).

Algorithm	t_1	t_2	t_3
PCA	0.000439	0.062	0.00557
KPCA	0.822	0.00404	0.521
SDAE-kNN	56.761	0.198	0.138

Table 8
Training time of one epoch for each sub-model in SDAE (s).

Algorithm	Training time	Network structure
DAE_1	0.0075	[33, 200, 33]
DAE_2	0.012	[200, 100, 200]
DAE_3	0.0065	[100, 50, 100]
DAE_4	0.004	[50, 30, 50]
SDAE	0.027	[33, 200, 100, 50, 30, 50, 100, 200, 33]

ing data is denoted as t_1, the time for statistic construction and control limit estimation as t_2, and the time for online detecting in fault 1 dataset as t_3. Table 7 lists these time for above three methods and the training time of one epoch for each stacked-model in SDAE is also shown in Table 8. It can be seen from the two tables that the model training time of SDAE-kNN method is a bit long, which is caused by the iterative optimization process. Although the proposed method takes a bit long time to train, it can be acceptable by the off-line modeling process. In addition, the proposed method has a short on-line detection time and it is much less than the KPCA method, which shows that the proposed method is effective for the actual online monitoring.

5. Conclusions

In this paper, a novel SDAE-kNN method has been proposed to build automated feature learning model for nonlinear process monitoring. The method relies on an unsupervised deep neural network called stacked denoising autoencoder for automatically capturing the robust and crucial features from the complicated and highly nonlinear process data. Building SDAE model is composed of two steps: a stacked pretraining step and a global fine-tuning step. These two steps ensure that the deep neural network converges to a better local optimum and learns better features. Meanwhile, the advantage of the neural network to approximate arbitrary nonlinear function is also fully utilized. For the purpose of detecting faults, the kNN rule is applied to two resulting spaces (feature and residual spaces) to construct the monitoring statistics. The results of simulation on a 5-dimensional nonlinear numerical system and the TE benchmark process both demonstrate the superior performance of the proposed method over state-of-the-art methods. As part of the future work, we will further extend the proposed method to non-Gaussian problem and batch process monitoring.

Acknowledgement

The authors gratefully acknowledge the support from the National Natural Science Foundation of China (61273161).

References

[1] L.H. Chiang, R.D. Braatz, E.L. Russell, Fault Detection and Diagnosis in Industrial Systems, Springer Science & Business Media, 2001.
[2] Z. Ge, Z. Song, F. Gao, Review of recent research on data-based process monitoring, Ind. Eng. Chem. Res. 52 (10) (2013) 3543–3562.
[3] J.V. Kresta, J.F. MacGregor, T.E. Marlin, Multivariate statistical monitoring of process operating performance, Can. J. Chem. Eng. 69 (1) (1991) 35–47.
[4] S. Joe Qin, Statistical process monitoring: basics and beyond, J. Chemom. 17 (8–9) (2003) 480–502.
[5] S.J. Qin, Survey on data-driven industrial process monitoring and diagnosis, Annu. Rev. Control 36 (2) (2012) 220–234.
[6] Z. Ge, Review on Data-Driven Modeling and Monitoring for Plant-Wide Industrial Processes, Chemometrics and Intelligent Laboratory Systems, 2017.
[7] S. Yin, S.X. Ding, A. Haghani, H. Hao, P. Zhang, A comparison study of basic data-driven fault diagnosis and process monitoring methods on the benchmark Tennessee Eastman process, J. Process Control 22 (9) (2012) 1567–1581.
[8] E.L. Russell, L.H. Chiang, R.D. Braatz, Data-Driven Methods for Fault Detection and Diagnosis in Chemical Processes, Springer Science & Business Media, 2012.
[9] Z. Ge, Z. Song, S.X. Ding, B. Huang, Data mining and analytics in the process industry: the role of machine learning, IEEE Access 5 (2017) 20590–20616.
[10] J. Huang, X. Yan, Gaussian and non-Gaussian double subspace statistical process monitoring based on principal component analysis and independent component analysis, Ind. Eng. Chem. Res. 54 (3) (2015) 1015–1027.
[11] S. Mahadevan, S.L. Shah, Fault detection and diagnosis in process data using one-class support vector machines, J. Process Control 19 (10) (2009) 1627–1639.
[12] N. Li, Y. Yang, Using semi-nonnegative matrix underapproximation for statistical process monitoring, Chemom. Intell. Lab. Syst. 153 (2016) 126–139.
[13] Q. Jiang, X. Yan, Weighted kernel principal component analysis based on probability density estimation and moving window and its application in nonlinear chemical process monitoring, Chemom. Intell. Lab. Syst. 127 (2013) 121–131.
[14] L.H. Chiang, E.L. Russell, R.D. Braatz, Fault diagnosis in chemical processes using fisher discriminant analysis, discriminant partial least squares, and principal component analysis, Chemom. Intell. Lab. Syst. 50 (2) (2000) 243–252.
[15] Q. Jiang, X. Yan, Just-in-time reorganized PCA integrated with SVDD for chemical process monitoring, AIChE J. 60 (3) (2014) 949–965.
[16] U. Kruger, S. Kumar, T. Littler, Improved principal component monitoring using the local approach? Automatica 43 (9) (2007) 1532–1542.
[17] Z. Ge, C. Yang, Z. Song, Improved kernel PCA-based monitoring approach for nonlinear processes, Chem. Eng. Sci. 64 (9) (2009) 2245–2255.
[18] J.-M. Lee, C. Yoo, S.W. Choi, P.A. Vanrolleghem, I.-B. Lee, Nonlinear process monitoring using kernel principal component analysis, Chem. Eng. Sci. 59 (1) (2004) 223–234.
[19] M.A. Kramer, Autoassociative neural networks, Comput. Chem. Eng. 16 (4) (1992) 313–328.
[20] D. Dong, T.J. McAvoy, Nonlinear principal component analysis-based on principal curves and neural networks, Comput. Chem. Eng. 20 (1) (1996) 65–78.
[21] Z. Ge, M. Zhang, Z. Song, Nonlinear process monitoring based on linear subspace and Bayesian inference, J. Process Control 20 (5) (2010) 676–688.
[22] Z. Ge, F. Gao, Z. Song, Two-dimensional Bayesian monitoring method for nonlinear multimode processes, Chem. Eng. Sci. 66 (21) (2011) 5173–5183.
[23] J.-H. Cho, J.-M. Lee, S.W. Choi, D. Lee, I.-B. Lee, Fault identification for process monitoring using kernel principal component analysis, Chem. Eng. Sci. 60 (1) (2005) 279–288.
[24] S.W. Choi, C. Lee, J.-M. Lee, J.H. Park, I.-B. Lee, Fault detection and identification of nonlinear processes based on kernel PCA, Chemom. Intell. Lab. Syst. 75 (1) (2005) 55–67.
[25] S.W. Choi, I.-B. Lee, Nonlinear dynamic process monitoring based on dynamic kernel PCA, Chem. Eng. Sci. 59 (24) (2004) 5897–5908.
[26] N. Li, Y. Yang, Ensemble kernel principal component analysis for improved nonlinear process monitoring, Ind. Eng. Chem. Res. 54 (1) (2014) 318–329.
[27] H. Yu, F. Khan, Improved latent variable models for nonlinear and dynamic process monitoring, Chem. Eng. Sci. (2017), http://dx.doi.org/10.1016/j.ces.2017.04.048.
[28] L. Luo, S. Bao, J. Mao, D. Tang, Nonlinear process monitoring using data-dependent kernel global-local preserving projections, Ind. Eng. Chem. Res. 54 (44) (2015) 11126–11138.
[29] P. Vincent, H. Larochelle, I. Lajoie, Y. Bengio, P.-A. Manzagol, Stacked denoising autoencoders: learning useful representations in a deep network with a local denoising criterion, J. Mach. Learn. Res. 11 (December) (2010) 3371–3408.
[30] P. Vincent, H. Larochelle, Y. Bengio, P.-A. Manzagol, Extracting and composing robust features with denoising autoencoders, in: Proceedings of the 25th International Conference on Machine Learning, ACM, 2008, pp. 1096–1103.
[31] X. Glorot, A. Bordes, Y. Bengio, Domain adaptation for large-scale sentiment classification: a deep learning approach, Proceedings of the 28th International Conference on Machine Learning (ICML-11) (2011) 513–520.
[32] X. Lu, Y. Tsao, S. Matsuda, C. Hori, Speech Enhancement Based on Deep Denoising Autoencoder, Interspeech (2013) 436–440.
[33] Q. Wang, W. Guo, K. Zhang, A.G. Ororbia II, X. Xing, X. Liu, C.L. Giles, Adversary resistant deep neural networks with an application to Malware detection, in: Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, ACM, 2017, pp. 1145–1153.
[34] G.E. Hinton, R.R. Salakhutdinov, Reducing the dimensionality of data with neural networks, Science 313 (5786) (2006) 504–507.
[35] H. Wang, X. Shi, D.-Y. Yeung, Relational stacked denoising autoencoder for tag recommendation, AAAI (2015) 3052–3058.
[36] J. Friedman, T. Hastie, R. Tibshirani, The Elements of Statistical Learning, vol. 1, Springer Series in Statistics Springer, Berlin, 2001.
[37] Q.P. He, J. Wang, Fault detection using the k-nearest neighbor rule for semiconductor manufacturing processes, IEEE Trans. Semicond. Manuf. 20 (4) (2007) 345–354.

[38] E. Parzen, On estimation of a probability density function and mode, Ann. Math. Stat. 33 (3) (1962) 1065–1076.
[39] E. Martin, A. Morris, Non-parametric confidence bounds for process performance monitoring charts, J. Process Control 6 (6) (1996) 349–358.
[40] A.R. Mugdadi, I.A. Ahmad, A bandwidth selection for kernel density estimation of functions of random variables, Comput. Stat. Data Anal. 47 (1) (2004) 49–62.
[41] J.J. Downs, E.F. Vogel, A plant-wide industrial process control problem, Comput. Chem. Eng. 17 (3) (1993) 245–255.
[42] P.R. Lyman, C. Georgakis, Plant-wide control of the Tennessee Eastman problem, Comput. Chem. Eng. 19 (3) (1995) 321–331.
[43] J. Huang, X. Yan, Related and independent variable fault detection based on KPCA and SVDD, J. Process Control 39 (2016) 88–99.
[44] Z. Ge, Z. Song, Nonlinear probabilistic monitoring based on the Gaussian process latent variable model, Ind. Eng. Chem. Res. 49 (10) (2010) 4792–4799.